

Dossiê técnico atual do projeto

Arquitetura, regras, cartas, arena, decks, economia, ranqueadas, recompensas, bots dinâmicos, multiplayer, Android/Windows, testes e guia de continuidade.

Produto	Arcane Duel
Diretório auditado	C:\hm\CARD12
Versão	1.2.0
Unity	6000.5.0f1 / URP 17.5.0
Autoridade de regras	ygopro-core API 11.0 + CardScripts/Lua
Plataformas	Windows x64 e Android ARM64
Snapshot	11 de agosto de 2026

Documento preparado para transferência de contexto a outro ChatGPT/Codex. Ele separa claramente o que está implementado, o que foi validado e o que permanece como evolução planejada.



1. Resumo executivo

Arcane Duel é um jogo de duelo de cartas em Unity com login, menu principal, editor de decks, loja, coleção, economia, arena, partidas contra bot, ranqueadas e multiplayer privado 1v1. O ygopro-core é a única autoridade das regras; a Unity apresenta o estado, coleta uma intenção e responde somente a prompts legais emitidos pelo Core.

Regra arquitetural central
Nenhum bot, botão, animação ou objeto da cena pode inventar uma jogada. Core/Lua valida regras e efeitos; C# escolhe uma alternativa legal, apresenta o resultado e persiste metadados de produto.

Área	Estado no snapshot
Duelo	Loop local jogável e motor oficial integrado; prompts, fases, Correntes, batalha e vencedor vêm do Core.
Cartas	Catálogo por ID oficial, dados compilados, scripts Lua e arte; RuntimeId diferencia cópias físicas.
Decks/economia	Editor, deck ativo, loja, pacotes, inventário, moedas e ledger idempotente.
Ranqueadas	Nove elos, 0–200 PE, recibos determinísticos, escudo de promoção e recompensas de moeda por elo.
Bots	33 perfis estáveis, cinco níveis, personalidade/arquétipo, persistência de PE e decisão apenas entre escolhas legais.
Multiplayer	Host autoritativo, Relay/NGO, snapshots filtrados, resync e compatibilidade versionada.
Android	ARM64, minSdk 26, IL2CPP, runtime nativo ocgcore e automatizador de APK em D:\APK.

2. Princípios e arquitetura

Separação obrigatória entre domínio autoritativo e apresentação.

Camada	Responsabilidade	Exemplos
Unity / apresentação	Cena, UI, input, animação, áudio e projeção visual.	Arena, menus, CardView, trays
Agentes	Selecionar resposta exata entre alternativas legais.	TacticalOpponentAgent, BotDecisionService
Ponte C#	Marshaling, callbacks, protocolo, handles e snapshot.	OcgDuelEngine, Interop, Protocol
ygopro-core	Legalidade, custos, alvos, Correntes, batalha, RNG e resultado.	Biblioteca nativa Windows/Android
CardScripts/Lua	Efeitos oficiais por ID de carta.	StreamingAssets/Ygo/Scripts
BabelCDB	Metadados e texto oficial/localizado.	cards.bin, card-texts.json
Rede	Transporte, autoridade, comandos, ACK e resync.	MPS, Relay DTLS, NGO

- OfficialCardId identifica a definição; RuntimeId identifica cada cópia física.
- Mão, Deck, cartas baixadas e escolhas privadas são filtrados por perspectiva.
- O host ou o modo local possui a única instância autoritativa do Core.
- Animações podem atrasar a apresentação, nunca a confirmação das regras.
- Bots reutilizam a mesma fronteira de prompts/choices destinada ao multiplayer.

Fluxo lógico

Comando até a tela
Jogador/bot → intenção → prompt legal do Core → escolha exata → resolução no ygopro-core → mensagens tipadas → estado de apresentação → UI/animação/rede.

3. Cenas e fluxo do jogador

Cena	Uso
MainMenu	Hub de duelo, ranqueadas, decks, loja, opções e multiplayer.
DeckEditor	Consulta de coleção e construção/seleção do deck ativo.
DuelArena	Arena principal autorada para local, bot e online.
Duel	Apresentação Core-only/compatibilidade.
CardLab	Validação isolada de cartas e apresentação.

Fluxo local e contra bot

- MainMenu → DUELAR → casual com escolha de bot/deck ou ranqueado com matchmaking automático por PE → DuelArena.
- O casual não altera PE. No ranqueado o jogador não escolhe dificuldade; a busca amplia a janela de PE apenas quando necessário e sela snapshots dos dois participantes.
- O bot recebe somente prompts destinados ao assento oponente e responde após atraso visual configurado.
- Ao fim autoritativo, o evento DuelCompleted gera recibos ranqueados idempotentes para ambos.

Fluxo online

- Sala privada ou fila ranqueada; compatibilidade verifica app, protocolo, Core, scripts, banco e banlist.
- Somente o host executa o Core; o convidado transmite resposta ao prompt e recebe estado confirmado.
- Loading, SceneReady, SnapshotApplied e BeginDuel impedem ações antes da convergência.
- CoreMessage.Win é terminal; a UI não infere vencedor apenas por LP.

4. Motor de regras e mecânicas de duelo

Grupo	Cobertura
Fluxo	START, turnos, fases, prioridade, espera, WIN e encerramento.
Movimento	DRAW, MOVE, SHUFFLE, SET, POS_CHANGE, pilhas e zonas.
Invocações	Normal, Tributo, Ritual, Fusion, Synchro, Xyz e Link conforme scripts/Core.
Correntes	Ativação, respostas, ordem LIFO, resolução e encerramento.
Escolhas	Cartas, soma, ordem, sim/não, posição, zona, opção, tipo, atributo e número.
Batalha	Declaração, alvo, replay, cálculo, dano, destruição e resultado.
Estado	Deck, mão, campo, Cemitério, banidas, Extra, overlays, posição e informação oculta.

Fidelidade Yu-Gi-Oh!

Destruir, oferecer como Tributo, descartar, pagar custo, devolver, banir e usar como material não são sinônimos. A origem e o destino vêm das mensagens do Core; a apresentação não falsifica eventos.

Identidade e consistência

- Uma carta ocupa uma única localização autoritativa; a cena pode reconstruí-la por snapshot.
- Cartas iguais continuam independentes pelo RuntimeId.
- Prompts antigos são recusados por RequestId/versão; cliques visuais nunca sobrevivem a uma nova decisão.
- Efeitos oficiais permanecem em Lua. Correções visuais são feitas no decoder/projector/presenter.

5. Cartas, catálogo e decks

As cartas são identificadas pelo ID oficial. Dados, scripts e arte formam um pacote sincronizado. A existência de uma imagem não prova que o efeito funciona; a prova é um cenário executado pelo Core com resposta legal e resultado esperado.

Fonte	Finalidade
Assets/Cards/CardCatalog.asset	Catálogo de apresentação e associação de arte/metadados.
Assets/StreamingAssets/Ygo/Data/cards.bin	Banco compilado consumido em runtime.
Assets/StreamingAssets/Ygo/Scripts	CardScripts oficiais/selecionados.
Assets/StreamingAssets/Ygo/Art	Arte usada no runtime.
ThirdParty	Snapshots fixados do Core, CardScripts e BabelCDB.
DeckRepository	Decks, coleção, deck ativo, inventário, moedas, recibos e migração.

Regras de extensão

- Cadastrar sempre pelo ID oficial; nomes traduzidos são apresentação.
- Não deduzir categoria por cor da imagem.
- Não duplicar um efeito em C# se já existe no CardScript/Core.
- Todo novo lote deve validar banco, script, arte, deck legal e cenário de efeito.
- DeckPowerScore de bot deve ser publicado pelo pipeline Editor/CI; não usar ATK, raridade ou texto como atalho em runtime.

6. Arena, HUD e apresentação

Sistema	Comportamento
Campo	Zonas autoradas e cartas projetadas pelo estado confirmado.
Mão	Leque responsivo, hover/seleção, RuntimeId preservado e reorganização dinâmica.
Inspector	Arte, nome, atributo, nível/rank/link, tipo, ATK/DEF e texto com rolagem.
Ações	Somente comandos legais do prompt atual; opções compactas e cancelamento quando permitido.
Trays	Escolha de cartas/zonas, minimização e informação privada por dono.
Pilhas	Deck, Extra, Cemitério e banidas com contadores e navegação permitida.
Animações	Compra, invocação, ativação e Correntes; preferências não alteram regras.

- Canvas e Safe Area suportam diferentes proporções de tela.
- Texturas e visuais podem ser substituídos sem trocar IDs de zona ou carta.
- Objetos de cena são apresentação; não entram no hash autoritativo.
- No Android, evitar geração de sprites/atlases e self-play durante a partida.

7. Economia, loja e persistência

Recurso	Implementação
Moedas	Saldo no DeckRepository; alterações registradas por transação.
Loja	Catálogo de pacotes, preço, abertura e inventário persistente.
Idempotência	transactionId impede compra, prêmio ou recompensa duplicada.
Atomicidade	Save temporário/substituição; falha restaura snapshot quando aplicável.
Ranqueadas	Recibos limitados e versionados; o estado é validado antes do commit.
Bots	bots.json separado em persistentDataPath/ArcaneArena, com troca via arquivo temporário.

Proteção contra duplicação
Recompensas de elo usam rank-promotion:v1:{profileId}:{tier}. Carregar o save, repetir o resultado ou reabrir a tela não paga novamente.

Migração de saves

- Schema atual: 8.
- Perfis antigos preservam decks, cartas, moedas e identidade.
- A inclusão de Bronze remapeia apenas o enum do escudo de promoção quando necessário.
- Recompensas anteriores faltantes são concedidas até o elo atual uma única vez.

8. Ranqueadas, Pontos de Elo e recompensas

Elo	Faixa de PE	Recompensa
Madeira	0–24	40 moedas
Pedra	25–49	60 moedas
Ferro	50–74	80 moedas
Bronze	75–99	120 moedas
Prata	100–124	180 moedas
Ouro	125–149	300 moedas
Platina	150–174	500 moedas
Diamante	175–199	1.000 moedas
Grão-Mestre	200	1.875 moedas

- RankRules v2 e hash rank-v2-nine-leagues-2026-08-11.
- PE permanece no intervalo 0–200.
- Resultado ranqueado produz recibo com matchId, jogador, oponente, PE anterior/novo, tiers, escudo, versão e hash.
- O resultado é recalculado localmente antes do commit; recibo obsoleto ou incompatível é recusado.
- A interface mostra a recompensa do elo atual e a próxima recompensa disponível.

Madeira também é recompensada
Um perfil novo ou migrado recebe as 40 moedas de Madeira no primeiro carregamento válido. O ledger impede uma segunda concessão.

9. Sistema de bots dinâmicos

Implementação baseada na especificação fornecida pelo usuário.

Nível	ε	Temperatura	Máx. subótimo	Top-k	PE inicial	Janela de poder
Iniciante	0,22	1,35	18%	2	25	20–45
Intermediário	0,12	0,95	10%	3	65	35–60
Avançado	0,06	0,65	5%	4	105	50–75
Especialista	0,025	0,40	2%	5	145	65–88
Mestre	0,005	0,18	0%	todas	175	78–100

Como o bot decide

- Recebe DuelPrompt e DuelPresentationState; nunca consulta GameObject para legalidade.
- A política tática existente pontua escolhas; personalidade ajusta preferências, não cria ações.
- Epsilon/temperatura/top-k controlam variedade e taxa de decisão subótima.
- Seed persistida + RequestId + sequência geram comportamento reproduzível.
- Prompts complexos reutilizam o resolvedor tático seguro; prompts unitários aceitam somente ChoiceIndex emitido pelo Core.

Grão-Mestre não é nível de IA
Os cinco níveis terminam em Mestre. Grão-Mestre é exclusivamente um elo competitivo.

10. Catálogo dos 33 bots — parte 1

ID	Nome	Nível	Personalidade	Arquétipo
BOT_001	Fraudinha	Iniciante	Oportunista	Aggro / Tempo
BOT_002	Baroquinha	Iniciante	Agressivo	Aggro
BOT_003	Funciondicero	Iniciante	Casual	Midrange
BOT_004	Ordelei	Iniciante	Defensivo	Control
BOT_005	Strada	Iniciante	Agressivo	Combo
BOT_006	Biólogo	Iniciante	Curioso	Toolbox
BOT_007	Ana	Iniciante	Equilibrado	Midrange
BOT_008	Viniado	Intermediário	Agressivo	Aggro / Combo
BOT_009	Coutinho	Intermediário	Equilibrado	Midrange
BOT_010	Talason	Intermediário	Calculista	Control
BOT_011	Volpi	Intermediário	Oportunista	Tempo
BOT_012	Márcia	Intermediário	Defensivo	Control
BOT_013	Jão	Intermediário	Agressivo	Aggro
BOT_014	Fisheballcat	Intermediário	Criativo	Toolbox
BOT_015	Lucas Gay	Avançado	Oportunista	Combo / Tempo
BOT_016	Joseph Rosewel	Avançado	Calculista	Control
BOT_017	But-2-eno	Avançado	Equilibrado	Midrange / Combo

11. Catálogo dos 33 bots — parte 2

ID	Nome	Nível	Personalidade	Arquétipo
BOT_018	Aura	Avançado	Defensivo	Control / Tempo
BOT_019	Juan	Avançado	Agressivo	Aggro / Combo
BOT_020	Mario	Avançado	Equilibrado	Midrange
BOT_021	Electrolux	Avançado	Oportunista	Toolbox / Tempo
BOT_022	Pablo Escobar	Especialista	Agressivo	Combo
BOT_023	2,2,4-Trimetilpentano	Especialista	Calculista	Toolbox / Combo
BOT_024	Dr. Júlio	Especialista	Calculista	Control
BOT_025	Shalon	Especialista	Defensivo	Control / Tempo
BOT_026	Mustang	Especialista	Agressivo	Aggro / Combo
BOT_027	Seixas	Especialista	Equilibrado	Midrange / Toolbox
BOT_028	Exterminator	Mestre	Agressivo	Combo / OTK
BOT_029	Kindred	Mestre	Calculista	Control / Tempo
BOT_030	Boy Pinto	Mestre	Oportunista	Combo / Toolbox
BOT_031	Florêncio	Mestre	Defensivo	Control
BOT_032	Luffitoro	Mestre	Equilibrado	Midrange / Combo
BOT_033	Super Choque	Mestre	Agressivo	Tempo / Combo

IDs são permanentes

Nome exibido pode ser localizado; botId não muda. Persistência, ranqueadas, logs e futuras temporadas devem referenciar BOT_001...BOT_033.

12. Persistência e ranqueadas dos bots

Campo	Uso
botId	Identidade estável do perfil.
rankedPoints	PE atual do bot, limitado pelas mesmas regras do jogador.
stateVersion	Deteção de snapshot obsoleto.
seasonId	Temporada associada ao estado.
deckVariantId	Variante de deck selecionada/publicada.
wins / losses	Histórico competitivo resumido.
processedRankTransactions	Últimas 256 chaves para idempotência.

Partida ranqueada contra IA

- Antes do duelo, jogador e bot fornecem RankPlayerSnapshot selado.
- O mesmo RankPointService calcula os dois recibos.
- O jogador grava no DeckRepository; o bot grava em bots.json.
- Repetir DuelCompleted não duplica PE porque transactionId é estável.
- Casual e demonstração não alteram PE.

Escopo ainda planejado

- Preenchimento automático de fila online por bot após timeout exige integração de serviço/servidor e não foi simulado no cliente.
- DeckPowerScore deve ser calculado por self-play no Editor/CI e publicado como dado; o runtime apenas respeita a janela.
- Temporadas futuras precisam de política explícita de reset/carry-over e migração versionada.

13. Multiplayer e crossplay

Mecanismo	Garantia
Autoridade	Somente host executa OcgDuelEngine.
Comandos	requestId/commandId, jogador, sequência e versão são validados.
Privacidade	Snapshots filtrados; mão, Deck, sets e prompts privados não vazam.
Sincronização	Hash, ACK, resync e snapshot recuperam divergências transportáveis.
Compatibilidade	App/Core/scripts/banco/banlist/protocolo precisam coincidir.
Transporte	MPS Sessions, Relay DTLS, NGO e fragmentação para payload grande.
Bot	Loopback local usa a mesma fronteira de prompt/choice; bot não é cliente autoritativo.

Matriz física obrigatória
Automação não substitui PC↔PC, Android↔Android e PC↔Android com duas contas, redes reais, pause/resume, perda temporária, reconexão, resultado e segunda partida.

Não adicionar

- Segundo NetworkManager.
- Cálculo de regra no cliente.
- Uso de posição de tela, nome ou instanceId como identidade de gameplay.
- Seed, mão ou escolha privada enviada ao assento errado.

14. Windows e Android

Item	Configuração
Unity	6000.5.0f1 (mesma revisão em desenvolvimento e build).
Android	APK ARM64, IL2CPP, minSdk 26, target automático/SDK instalado.
Runtime nativo	Assets/Plugins/Android/arm64-v8a/libcgccore.so.
Java/Gradle	OpenJDK 17.0.18+8, Gradle 9.1, Android Gradle Plugin 9.0.
SDK	Build Tools/Platform Tools 36.0.0 e plataformas instaladas pelo Unity Hub.
NDK	r27c (27.2.12479018).
Saída	D:\APK\ArcaneDuel-v1.2.0-arm64.apk

APK produzido e verificado

Arquivo: D:\APK\ArcaneDuel-v1.2.0-arm64.apk

Tamanho: 647.7 MiB

SHA-256: 255A1DD031BE6F5C1E4B54A4090042E2B8670B667772A7BD2D4792EF289D277B

Cuidados de desempenho

- Não gerar decks, scores, bancos ou sprites durante o duelo.
- Reutilizar visualizações e reduzir alocações por prompt.
- ARM64/IL2CPP é o alvo Android de release; bibliotecas nativas devem ter a mesma ABI.
- Safe Area, touch e lifecycle Android devem preservar estado e bloquear input durante resync.

15. Testes e evidências desta atualização

Conjunto	Resultado	Cobertura principal
DynamicBotSystemEditModeTests	9/9	33 IDs, dificuldade, persistência e matchmaking por PE.
RankPointServiceEditModeTests	41/41	Bronze, faixas, deltas, GM, recompensas.
Economia focada	24/24	Ledger, autorização, migração schema 8 e Wood 40.
Total focado	74/74	Regressão direta dos recursos implementados.
Suite EditMode ampla	379/432	53 falhas pré-existentes/de conteúdo nativo ou visual; não atribuídas ao novo catálogo de bots.
Compilação C#	Sucesso	Unity 6000.5.0f1, Android target importado.

Gates recomendados antes de release público

- Executar novamente EditMode completo e classificar cada falha por baseline vs regressão.
- Smoke test de todos os níveis de bot e uma partida ranqueada por nível.
- Confirmar uma única recompensa por elo após reload e repetição de resultado.
- Executar matriz crossplay física e coletar Player.log/logcat.
- Instalar o APK em ao menos um Android ARM64 minSdk 26 e validar Core nativo.

16. O que foi implementado e o que não foi fingido

Requisito	Estado	Observação
33 perfis de bot	Implementado	IDs, níveis, personalidade, arquétipos e PE inicial.
Decisão legal	Implementado	Escolhe somente DuelChoice emitido pelo Core.
Persistência do bot	Implementado	bots.json atômico e versionado.
Bot casual	Implementado	Seleção de perfil/deck; sem PE.
Bot ranqueado	Implementado	Seleção automática por PE, memória curta, snapshots e recibos idempotentes.
Nove elos + prêmios	Implementado	40 a 1.875 moedas, visíveis na interface.
Human-first fill online	Planejado	Requer orquestração de fila/serviço; não criar autoridade falsa no cliente.
Self-play / DeckPowerScore	Planejado	Deve rodar no Editor/CI e publicar score certificado.
Temporadas completas	Base pronta	seasonId existe; reset e agenda exigem política de produto.
IA treinada	Não	Sistema é heurístico, controlado e reproduzível.

Transparência técnica

Uma escolha visual funcionando não prova uma regra. Uma carta/feature é considerada pronta somente quando o Core aceita, resolve, produz o estado correto e os testes confirmam os casos legal e ilegal.

17. Mapa de arquivos para continuidade

Caminho	Responsabilidade
Assets/Game/Runtime/Bots/DynamicBotSystem.cs	Catálogo, dificuldade, estado, seleção e decisão dos bots.
Assets/Game/Runtime/TacticalOpponentPolicy.cs	Pontuação tática reutilizada pelo agente.
Assets/Game/Runtime/DuelArenaController.cs	Fronteira arena↔Core, prompts, bot e conclusão do duelo.
Assets/Game/Runtime/Competitive/RankDomain.cs	Elos, PE, recibos, escudos e tabela de prêmios.
Assets/Scripts/Frontend/DeckRepository.Ranking.cs	Commit ranqueado e concessão idempotente de moedas.
Assets/Scripts/Frontend/GameFrontendBootstrap*.cs	Menus, seleção casual e matchmaking ranqueado automático.
Assets/Scripts/CardArenaBootstrap.cs	Composição/apresentação legada; não deve ganhar regras novas.
Assets/DuelEngine/Runtime	Interop, Core, protocolos, dados, scripts e estado.
Assets/Scripts/Multiplayer	Sessões, transporte, snapshots, loading e resultado.
Assets/Editor/AndroidBuildAutomation.cs	Build reproduzível Android em D:\APK.
Assets/Tests/EditMode	Regressões de bots, rank, economia e Core.

18. Procedimento operacional

Abrir e testar

- Abrir C:\hm\CARD12 com Unity 6000.5.0f1.
- Aguardar importação sem interromper; não abrir duas instâncias no mesmo projeto.
- Executar testes EditMode focados em bots, rank e economia.
- Testar bot casual e bot ranqueado com decks distintos e iguais.
- Conferir prêmio do elo, reload e ausência de duplicação.

Build Android

Menu: Card Game → Build → Android APK (D:\APK). Em CI/batchmode usar o método `ArcaneArena.EditorTools.AndroidBuildAutomation.BuildFromCommandLine`.

Checklist de distribuição

- APK contém `lib/arm64-v8a/libocgcore.so`.
- Versão e `versionCode` correspondem ao release.
- Nenhum erro `C#` ou `BuildFailedException` no log.
- Instalação e início em aparelho ARM64 minSdk 26.
- Duelo contra bot completa ao menos 10 turnos sem travar.
- Matriz crossplay executada antes de declarar o online pronto.

19. Como evoluir sem criar um segundo motor

Ao acrescentar uma carta, um bot ou uma regra, siga a ordem:

- 1. Verificar se o ygo-pro-core/CardScript já implementa a mecânica.
- 2. Garantir metadados e ID oficial no catálogo/banco.
- 3. Expor a decisão pelo protocolo tipado existente.
- 4. Fazer UI e bot consumirem somente escolhas legais.
- 5. Criar teste de ativação legal, ilegal, custo, alvo, resolução e Corrente.
- 6. Verificar informação oculta e perspectiva multiplayer.
- 7. Rodar offline, bot, host/cliente e build Android/Windows conforme risco.

Evitar crescimento monolítico

CardArenaBootstrap e GameFrontendBootstrap são grandes por legado. Novos sistemas devem entrar em serviços pequenos, testáveis e side-agnostic, deixando os bootstraps apenas como composição e apresentação.

Bot novo

- Adicionar perfil com botId estável e sem reutilizar ID.
- Escolher nível, personalidade e arquétipo; publicar variante de deck certificada.
- Não criar switch por nome de carta dentro do agente.
- Testar repetibilidade por seed e impossibilidade de escolher opção ilegal.

20. Contexto curto para outro ChatGPT/Codex

Este repositório é o Arcane Duel, em C:\hm\CARD12, Unity 6000.5.0f1, versão 1.2.0, Windows x64 e Android ARM64. O ygopro-core API 11.0 é a única autoridade de regras; CardScripts/Lua executa efeitos; BabelCDB fornece dados; Unity/C# cuida de interface, input, apresentação, bots, rede e persistência.

O projeto possui MainMenu, DeckEditor, DuelArena, loja, coleção, moedas, pacotes, ranqueadas, nove elos, recompensas idempotentes, 33 bots dinâmicos, duelo local e multiplayer host-autoritativo com Relay/NGO. Bots somente selecionam DuelChoice legal emitido pelo Core. O estado de bot é persistido por botId; ranqueadas contra IA usam snapshots e recibos iguais aos do jogador.

Antes de editar, leia git status e este PDF. Não crie outro motor, não reimplemente efeitos oficiais em C#, não use GameObject/nome/posição como identidade, não crie segundo NetworkManager e não trate IA como autoridade. Preserve informação oculta, idempotência, versão/hash de regras e compatibilidade crossplay. Mudanças devem ser incrementais, compiláveis, testadas e documentadas.

Fontes locais principais

- ProjectSettings/ProjectVersion.txt, ProjectSettings.asset e EditorBuildSettings.asset.
- Assets/Game/Runtime/ProjectIdentity.cs e runtime de bots/rank.
- Assets/DuelEngine/Runtime e Assets/Scripts/Multiplayer.
- Docs/MULTIPLAYER_TEST_REPORT.md, AUDIT_RANK_SYSTEM.md e relatórios de conteúdo.
- Especificacao_Sistema_Bots_Dinamicos_Elos_Temporadas_Arcane_Duel_Codex.pdf.